# Knowledge Organiser — 2.3 Algorithms

*OCR A Level Computer Science H446, Component 02, Section 2.3.1 — Algorithms.*

## Big O complexity classes

| Class | Name | Growth as n increases | Example |
|---|---|---|---|
| **O(1)** | Constant | No change with n | `array[i]`; stack push/pop |
| **O(log n)** | Logarithmic | Doubling n adds one step | Binary search |
| **O(n)** | Linear | Doubling n doubles time | Linear search; single loop |
| **O(n log n)** | Linearithmic | Slightly worse than linear | Merge/quick sort (average) |
| **O($n^2$)** | Polynomial (quadratic) | Doubling n quadruples time | Bubble/insertion sort; nested loops |
| **O($2^n$)** | Exponential | One extra item doubles time | Naïve recursive Fibonacci; brute-force TSP |

- Big O = **worst-case** growth of **time** (operations) or **space** (memory) vs input size n.
- Drop constants & lower-order terms: `3n² + 5n + 17` → **O($n^2$)** (dominant term wins).
- **Tractable** = solvable in polynomial time or better; **intractable** = only exponential (O($2^n$)) or worse.

## Standard algorithm complexities (VERIFIED)

| Algorithm | Best | Average | Worst | Space | Stable? |
|---|---|---|---|---|---|
| **Linear search** | O(1) | O(n) | O(n) | O(1) | — |
| **Binary search** | O(1) | O(log n) | O(log n) | O(1) iter / O(log n) rec | — |
| **Bubble sort** | O(n)* | O($n^2$) | O($n^2$) | O(1) | Yes |
| **Insertion sort** | O(n) | O($n^2$) | O($n^2$) | O(1) | Yes |
| **Merge sort** | O(n log n) | O(n log n) | O(n log n) | O(n) | Yes |
| **Quick sort** | O(n log n) | O(n log n) | **O($n^2$)** | O(log n) | No |

* Bubble best case O(n) only **with a swap flag** on already-sorted data; otherwise O($n^2$).

## How each one works (one line)

| Algorithm | How it works |
|---|---|
| **Linear search** | Check each element in turn from the start until found or list ends. Works on **unsorted** data. |
| **Binary search** | Check middle of a **sorted** list; discard the half that cannot contain target; repeat (halves space each pass). |
| **Bubble sort** | Repeatedly compare adjacent pairs and swap if out of order; largest "bubbles" to end each pass. |
| **Insertion sort** | Build a sorted front section; insert each next element into place by shifting larger ones right. |
| **Merge sort** | Divide-and-conquer: recursively split in half until single items, then merge sublists in order. |
| **Quick sort** | Divide-and-conquer: pick a pivot, partition (smaller left / larger right), recursively sort each side. |

> **Binary search precondition: the list MUST be sorted.** Quote this in every answer.

## Data structure operation complexities

| Structure | Operations | Complexity |
|---|---|---|
| **Stack** (LIFO) | push / pop / peek / isEmpty | O(1) |
| **Queue** (FIFO) | enQueue (rear) / deQueue (front) | O(1) |
| **Array** | access by index | O(1); insert/delete shifts → O(n) |
| **Linked list** | traverse / search | O(n); insert/delete given node → O(1) (re-point) |
| **Binary search tree** | search / insert | O(log n) avg (balanced), O(n) worst (degenerate); traversal O(n) |

- Circular queue uses `MOD` to wrap pointers and reuse freed space.
- **DFS uses a stack** (or recursion); **BFS uses a queue**.

## Tree traversals

Example tree: root 50; 30 (L) with children 20,40; 70 (R) with children 60,80.

| Traversal | Order | Output | Use |
|---|---|---|---|
| **Pre-order** | Node → Left → Right | 50, 30, 20, 40, 70, 60, 80 | Copy a tree; prefix expressions |
| **In-order** | Left → Node → Right | 20, 30, 40, 50, 60, 70, 80 | **Ascending sorted** output from a BST |
| **Post-order** | Left → Right → Node | 20, 40, 30, 60, 80, 70, 50 | Delete a tree; evaluate RPN (postfix) |
| **Breadth-first (BFS)** | Level by level, L→R (uses a **queue**) | 50, 30, 70, 20, 40, 60, 80 | Shortest path in unweighted graph |

- Mnemonic — position of **Node**: **Pre** = first, **In** = middle, **Post** = last.
- Post-order is the **depth-first** traversal the spec names explicitly.

## Path-finding: Dijkstra vs A*

| | **Dijkstra** | **A*** |
|---|---|---|
| Cost function | g(n) only | **f(n) = g(n) + h(n)** |
| Heuristic? | No (uninformed) | **Yes** (informed) |
| Search shape | Outward in all directions | Directed toward the goal |
| Typical use | Shortest path to **all** nodes | Shortest path to **one** goal |
| Efficiency | Explores more nodes | Usually explores **fewer** → faster |
| Optimality | Optimal (non-negative weights) | Optimal **if heuristic admissible** |

- **g(n)** = actual cost from start to n. **h(n)** = heuristic *estimate* of cost from n to goal (e.g. straight-line/Manhattan). **f(n)** = estimated total cost; always expand lowest f(n) next.
- **A* adds a heuristic estimate of the remaining distance to the goal** — that is the key difference.
- Heuristic must be **admissible** (never overestimates) for A* to guarantee the optimal path. If h(n)=0, A* reduces exactly to Dijkstra.
- Dijkstra: always visit the unvisited node with smallest tentative distance; relax neighbours; once visited, distance is final (don't update settled nodes).

## Must-know definitions

| **Term** | **Definition** |
|---|---|
| **Algorithm** | A precise, finite sequence of steps to solve a problem. |
| **Big O notation** | How an algorithm's time/space requirement grows with input size n (worst case). |
| **Time complexity** | Number of operations as a function of input size. |
| **Space complexity** | Memory required as a function of input size. |
| **Tractable / Intractable** | Solvable in polynomial time or better / only exponential or worse. |
| **Divide and conquer** | Break into subproblems, solve, combine (merge/quick sort, binary search). |
| **Pivot** | Element chosen in quick sort to partition the list. |
| **Stable sort** | Preserves relative order of equal elements. |
| **Heuristic** | A rule-of-thumb estimate; in A*, estimate h(n) of remaining cost to goal. |
| **Admissible heuristic** | One that never overestimates the true remaining cost → guarantees A* optimality. |
| **Traversal** | Visiting every node of a structure in a defined order. |

## Top exam reminders

- **Binary search is O(log n) and REQUIRES sorted data** — always state the precondition.

- **Quick sort worst case is O(n²)** (poor pivot / already-sorted); **merge sort guarantees O(n log n)** but uses **O(n) extra space** — classic time-vs-space trade-off.
- **Always justify** a Big O, e.g. "O(n²) because of the nested loop — each of n passes makes up to n comparisons."
- **DFS uses a stack; BFS uses a queue.** Don't mix them up.
- **In-order traversal of a BST outputs values in ascending order**; bubble sort best case is O(n) **only with a swap flag**.